Read me assignment 11

- Added 3 new variables to variables.tf, the key_name which is the EC2 key pair name used to SSH into all instances, ubuntu_ami which is the ami id for the 3 Ubuntu instances, amazon_ami which is the ami id for the 3 Amazon linux instances

```
maryamtendo — nano terraform/variables.tf — 80×24
  UW PICO 5.09                    File: terraform/variables.tf

  description = "Your IP address for SSH access to bastion"
}

variable "instance_type" {
  description = "EC2 instance type"
  default     = "t2.micro"
}

variable "key_name" {
  description = "EC2 Key Pair name"
}

variable "ubuntu_ami" {
  description = "Ubuntu 22.04 AMI ID"
}

variable "amazon_ami" {
  description = "Amazon Linux 2 AMI ID"
}

^G Get Help   ^O WriteOut   ^R Read File ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit       ^J Justify    ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- Updated terraform.tfvars to add the new variable values and retrieved the Ubuntu and linux 2 ami ids directly from AWS and then updated my_ip to reflect the current IP address.

```
maryamtendo — nano terraform/terraform.tfvars — 80×24
  UW PICO 5.09            File: terraform/terraform.tfvars            Modified

region        = "us-east-1"
ami_id        = "ami-0a4ac2b65a9e38f80"
my_ip         = "107.127.18.9/32"
instance_type = "t2.micro"
key_name      = "swarm-key"
ubuntu_ami    = "ami-00de3875b03809ec5"
amazon_ami    = "ami-0e10497160c48e829"

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- Updated main.tf replaced the 6 private instances block with 3 Ubuntu instances, 3 linux instances, and 1 ansible. Each group uses its own ami  and is tagged with the operating system type so ansible can target them separately. All instances are placed in the public subnet so the ansible controller can reach them directly too.

```
UW PICO 5.09                                File: terraform/main.tf

  user_data = <<-USERDATA
    #!/bin/bash
    echo "Port 443" >> /etc/ssh/sshd_config
    systemctl restart sshd
  USERDATA

  tags = {
    Name = "bastion-host"
  }
}

# 3 Ubuntu instances
resource "aws_instance" "ubuntu" {
  count                  = 3
  ami                    = var.ubuntu_ami
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = module.vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.bastion_sg.id]

  tags = {
    Name = "ubuntu-${count.index + 1}"
    OS   = "ubuntu"
  }
}

# 3 Amazon Linux instances
resource "aws_instance" "amazon" {
  count                  = 3
  ami                    = var.amazon_ami
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = module.vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.bastion_sg.id]

  tags = {
    Name = "amazon-${count.index + 1}"
    OS   = "amazon"
  }
}

# Ansible controller
resource "aws_instance" "ansible_controller" {
  ami                    = var.ubuntu_ami
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = module.vpc.public_subnets[0]
  vpc_security_group_ids = [aws_security_group.bastion_sg.id]

  tags = {
    Name = "ansible-controller"
  }
}

^G Get Help    ^O WriteOut    ^R Read File   ^Y Prev Pg     ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is    ^V Next Pg     ^U UnCut Text  ^T To Spell
```

- Updated outputs.tf, so i removed the old private instances output since those were replaced. Added outputs for the 3 Ubuntu instance IPs, 3 linux instance IPs, and the ansible controller IP so they are printed to the terminal after terraform apply and can be

used to build the ansible inventory.

```
  UW PICO 5.09                    File: terraform/outputs.tf                    Modified

output "bastion_public_ip" {
  description = "Public IP of the bastion host"
  value       = aws_instance.bastion.public_ip
}

output "ubuntu_instance_ips" {
  description = "Public IPs of the 3 Ubuntu instances"
  value       = aws_instance.ubuntu[*].public_ip
}

output "amazon_instance_ips" {
  description = "Public IPs of the 3 Amazon Linux instances"
  value       = aws_instance.amazon[*].public_ip
}

output "ansible_controller_ip" {
  description = "Public IP of the Ansible controller"
  value       = aws_instance.ansible_controller.public_ip
}

output "vpc_id" {
  description = "VPC ID"
  value       = module.vpc.vpc_id
}

^G Get Help    ^O WriteOut    ^R Read File   ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is    ^V Next Pg    ^U UnCut Text  ^T To Spell
```

- Ran terraform apply and terraform provisioned 27 resources successfully including the VPC, subnets, internet gateway, NAT gateway, security groups, and all 7 EC2 instances. The outputs printed the public IPs for all instances which I'll use to build the Ansible inventory.

```
maryamtendo — bash — 80×23

module.vpc.aws_route.private_nat_gateway[0]: Creating...
module.vpc.aws_route.private_nat_gateway[0]: Creation complete after 2s [id=r-rt
b-0951ec1aaac4618ee1080289494]

Apply complete! Resources: 27 added, 0 changed, 0 destroyed.

Outputs:

amazon_instance_ips = [
  "3.88.140.112",
  "18.234.147.5",
  "54.197.31.149",
]
ansible_controller_ip = "52.91.112.130"
bastion_public_ip = "54.208.202.63"
monitoring_private_ip = "10.0.2.205"
ubuntu_instance_ips = [
  "34.229.157.186",
  "98.91.211.96",
  "98.94.65.28",
]
vpc_id = "vpc-0649ff8d27781d746"
bash-3.2$
```

- Created ansible/inventory.ini and this I  defined the inventory file with two groups, amazon and ubuntu each of these containing 3 instances. Each host is assigned their default user (ubuntu for Ubuntu instances, ec2-user for Amazon Linux) and the SSH key used throughout this project. The Ips I used  were those I had taken directly from the terraform apply output.

```
maryamtendo — nano ansible/inventory.ini — 80×23

  UW PICO 5.09              File: ansible/inventory.ini              Modified

[ubuntu]
34.229.157.186 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-ke$
98.91.211.96 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-key.$
98.94.65.28 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-key.p$

[amazon]
3.88.140.112 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-ke$
18.234.147.5 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-ke$
54.197.31.149 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-k$

[all:vars]
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

PLAYBOOK

- Created ansible/playbook.yml  and in thos I wrote a playbook targeting all 6 instances. It uses the sudo apt module to update Ubuntu instances and the yum module to update linux instances. It then verifies docker is installed by running docker --version and reports disk usage using df -h on each instance. Output is printed.

```
UW PICO 5.09                    File: ansible/playbook.yml                    Modified

---
- name: Configure all EC2 instances
  hosts: all
  become: yes

  tasks:
    - name: Update packages on Ubuntu
      apt:
        update_cache: yes
        upgrade: yes
      when: ansible_os_family == "Debian"

    - name: Update packages on Amazon Linux
      yum:
        name: "*"
        state: latest
      when: ansible_os_family == "RedHat"

    - name: Check Docker is installed and get version
      command: docker --version
      register: docker_version

    - name: Print Docker version
      debug:
        msg: "{{ docker_version.stdout }}"

    - name: Report disk usage
      command: df -h
      register: disk_usage

    - name: Print disk usage
      debug:
        msg: "{{ disk_usage.stdout_lines }}"

^G Get Help    ^O WriteOut    ^R Read File    ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is     ^V Next Pg    ^U UnCut Text  ^T To Spell
```

- When trying to ssh into the ansible controller, the connection kept timing out on port 22. This is the same issue encountered in Assignment 9 where my network blocks outbound port 22. I fixed it the same way by adding a user_data block to all instances that configures SSH to run on port 443 instead, and opening port 443 in the security group. Also added associate_public_ip_address = true to all instances so they get public IPs.

Connected successfully after.

```
maryamtendo — bash — 80×23

bash-3.2$ scp -i ~/.ssh/swarm-key.pem ~/.ssh/swarm-key.pem ubuntu@52.91.112.130
~/.ssh/
ssh: connect to host 52.91.112.130 port 22: Operation timed out
scp: Connection closed
bash-3.2$
```

```
maryamtendo — ubuntu@ip-10-0-1-113: ~ — ssh -i ~/.ssh/swarm-key.pe...

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-113:~$
```

- SSH'd into the ansible controller and ran sudo apt update followed by sudo apt install -y ansible to install ansible. Installation completed successfully.

```
maryamtendo — ubuntu@ip-10-0-1-113: ~ — ssh -i ~/.ssh/swarm-key.pe...

Setting up ieee-data (20210605.1) ...
Setting up python3-dnspython (2.1.0-1ubuntu1) ...
Setting up python3-selinux (3.3-1build2) ...
Setting up python3-argcomplete (1.8.1-1.5) ...
Setting up python3-requests-ntlm (1.1.0-1.1) ...
Setting up python3-libcloud (3.2.0-2) ...
Setting up python3-netaddr (0.8.0-2) ...
Setting up python3-winrm (0.3.0-2) ...
Setting up ansible (2.10.7+merged+base+2.10.8+dfsg-1) ...
Processing triggers for man-db (2.10.2-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-113:~$
```

- Created inventory.ini with two groups, ubuntu and amazon, each with 3 instances using the IPs from the terraform apply output. Added ansible_port=443 under all vars since all instances are configured to run SSH on port 443, the same fix used in Assignment 9.

```
maryamtendo — ubuntu@ip-10-0-1-113: ~ — ssh -i ~/.ssh/swarm-key.pe...

  GNU nano 6.2                    /home/ubuntu/inventory.ini
[ubuntu]
98.88.24.148 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-key.>
100.48.97.122 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-key>
18.212.241.234 ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/swarm-ke>

[amazon]
100.31.207.82 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-k>
54.89.159.167 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-k>
13.218.76.79 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/swarm-ke>

[all:vars]
ansible_port=443
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

                              [ Read 13 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

- The playbook created targets all 6 instances and performs 3 tasks, updates and upgrades packages using apt for ubuntu and yum for linux , verifies Docker is installed by running docker --version and printing the result reports disk usage on each instance.

```
maryamtendo — ubuntu@ip-10-0-1-113: ~ — ssh -i ~/.ssh/swarm-key.pe...
  GNU nano 6.2                    /home/ubuntu/playbook.yml *
---
- name: Configure all EC2 instances
  hosts: all
  become: yes

  tasks:
    - name: Update packages on Ubuntu
      apt:
        update_cache: yes
        upgrade: yes
      when: ansible_os_family == "Debian"

    - name: Update packages on Amazon Linux
      yum:
        name: "*"
        state: latest
      when: ansible_os_family == "RedHat"

    - name: Check Docker is installed and get version

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

- Ran it and all 6 instances were unreachable because ansible was trying to connect on port 443. This was incorrect since the controller is inside AWS and communicates with the other instances within the same VPC, so port 22 is not blocked. Fixed by changing ansible_port from 443 to 22 in inventory.ini.

```
maryamtendo — ubuntu@ip-10-0-1-113: ~ — ssh -i ~/.ssh/swarm-key.pe...

PLAY [Configure all EC2 instances] *********************************************

TASK [Gathering Facts] *********************************************************
fatal: [98.88.24.148]: UNREACHABLE! => {"changed": false, "msg": "Failed to conn
ect to the host via ssh: ssh: connect to host 98.88.24.148 port 443: Connection
timed out", "unreachable": true}
fatal: [100.48.97.122]: UNREACHABLE! => {"changed": false, "msg": "Failed to con
nect to the host via ssh: ssh: connect to host 100.48.97.122 port 443: Connectio
n timed out", "unreachable": true}
fatal: [18.212.241.234]: UNREACHABLE! => {"changed": false, "msg": "Failed to co
nnect to the host via ssh: ssh: connect to host 18.212.241.234 port 443: Connect
ion timed out", "unreachable": true}
fatal: [100.31.207.82]: UNREACHABLE! => {"changed": false, "msg": "Failed to con
nect to the host via ssh: ssh: connect to host 100.31.207.82 port 443: Connectio
n timed out", "unreachable": true}
fatal: [54.89.159.167]: UNREACHABLE! => {"changed": false, "msg": "Failed to con
nect to the host via ssh: ssh: connect to host 54.89.159.167 port 443: Connectio
n timed out", "unreachable": true}
fatal: [13.218.76.79]: UNREACHABLE! => {"changed": false, "msg": "Failed to conn
ect to the host via ssh: ssh: connect to host 13.218.76.79 port 443: Connection
timed out", "unreachable": true}
```

- I encountered other errors, playbook failed on the docker version check because I hadn't installed it before. Fixed by adding two new tasks to the playbook to install docker before checking the version. Also, initially the inventory was using public IPs which were unreachable from the controller. Fixed by retrieving the private IPs of all instances using and updating the inventory to use private IPs instead, since the controller and all instances are in the same VPC and can communicate directly. Also changed ansible_port to 443 since all instances have SSH configured to run on that port. After all the corrections, packages updated, docker installed and verified and disk usage reported for each instance.

```
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
}
ok: [10.0.1.207] => {
    "msg": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "/dev/root       7.6G  2.8G  4.9G  37% /",
        "tmpfs           479M     0  479M   0% /dev/shm",
        "tmpfs           192M  904K  191M   1% /run",
        "tmpfs           5.0M     0  5.0M   0% /run/lock",
        "/dev/xvda15     105M  6.1M   99M   6% /boot/efi",
        "tmpfs            96M  4.0K   96M   1% /run/user/1000"
    ]
}
ok: [10.0.1.138] => {
    "msg": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "devtmpfs        473M     0  473M   0% /dev",
        "tmpfs           484M     0  484M   0% /dev/shm",
        "tmpfs           484M  468K  483M   1% /run",
        "tmpfs           484M     0  484M   0% /sys/fs/cgroup",
        "/dev/xvda1      8.0G  2.0G  6.1G  25% /",
        "tmpfs            97M     0   97M   0% /run/user/0",
        "tmpfs            97M     0   97M   0% /run/user/1000"
    ]
}
ok: [10.0.1.211] => {
    "msg": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "devtmpfs        473M     0  473M   0% /dev",
        "tmpfs           484M     0  484M   0% /dev/shm",
        "tmpfs           484M  468K  483M   1% /run",
        "tmpfs           484M     0  484M   0% /sys/fs/cgroup",
        "/dev/xvda1      8.0G  2.0G  6.1G  25% /",
        "tmpfs            97M     0   97M   0% /run/user/0",
        "tmpfs            97M     0   97M   0% /run/user/1000"
    ]
}
ok: [10.0.1.9] => {
    "msg": [
        "Filesystem      Size  Used Avail Use% Mounted on",
        "devtmpfs        473M     0  473M   0% /dev",
        "tmpfs           484M     0  484M   0% /dev/shm",
        "tmpfs           484M  468K  483M   1% /run",
        "tmpfs           484M     0  484M   0% /sys/fs/cgroup",
        "/dev/xvda1      8.0G  2.0G  6.1G  25% /",
        "tmpfs            97M     0   97M   0% /run/user/0",
        "tmpfs            97M     0   97M   0% /run/user/1000"
    ]
}

PLAY RECAP *********************************************************************
10.0.1.131                 : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
10.0.1.138                 : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
10.0.1.145                 : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
10.0.1.207                 : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
10.0.1.211                 : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
10.0.1.9                   : ok=7    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

ubuntu@ip-10-0-1-113:~$
```

**How to Run**

Must have terraform installed, AWS credentials configured (aws configure), SSH key pair named swarm-key downloaded to ~/.ssh/swarm-key.pem

**1. Provision the infrastructure**

bash
cd terraform
terraform init
terraform apply

Note the output IPs for the ansible controller and all instances.

**2. SSH into the ansible controller**

ssh -i ~/.ssh/swarm-key.pem -p 443 ubuntu@<ansible_controller_ip>

**3. Copy the SSH key to the controller** (from your local machine)

scp -i ~/.ssh/swarm-key.pem -P 443 ~/.ssh/swarm-key.pem ubuntu@<ansible_controller_ip>:~/.ssh/

**4. Install Ansible on the controller**

bash
```
sudo apt update
sudo apt install -y ansible
```

**5. Copy the inventory and playbook to the controller**

bash
```
scp -i ~/.ssh/swarm-key.pem -P 443 ansible/inventory.ini ubuntu@<ansible_controller_ip>:~/
scp -i ~/.ssh/swarm-key.pem -P 443 ansible/playbook.yml ubuntu@<ansible_controller_ip>:~/
```

**6. Run the playbook**

bash
```
ansible-playbook -i ~/inventory.ini ~/playbook.yml
```